BCSE308P Computer Networks Lab - Assignment 10

1. Course & Faculty Information

Course Code & Title: BCSE308P - Computer Networks Lab

Faculty: Prof. SREETHAR S

2. Project & Team Information

Project Title: Protocol Lab: An Interactive Computer Networks Simulator

Slot / Team: F1_Team34

Contributors:

• 24BCT0309 - SAVIR GHAI

3. GitHub Repository Submission

As per the requirements for Assignment 10, below is the link to the GitHub repository containing the complete project source code, simulation and experiment scripts, and all supporting documents:

Main GitHub Repository URL: https://github.com/savir18/CN

Direct Links to Required Files:

- Source Code:
  https://github.com/savir18/CN/tree/main/js
  HTML in f1_team34_index.html; styles in /css; tests in /tests; experiment runner in /scripts
- Network Simulation (.pkt):
  https://github.com/savir18/CN/blob/main/scripts/f1_team34_experiments.js
  Not applicable: Cisco Packet Tracer was not used. The simulation is a browser-based simulator; this is its 30-seed experiment runner.
- Screenshots:
  https://github.com/savir18/CN/tree/main/screenshots
- Project Report (PDF):
  https://github.com/savir18/CN/blob/main/docs/f1_team34_report.pdf
- Project Presentation:
  https://github.com/savir18/CN/blob/main/docs/f1_team34_presentation.pdf
  Editable version: docs/f1_team34_presentation.pptx
- Readme file:
  https://github.com/savir18/CN/blob/main/f1_team34_README.md

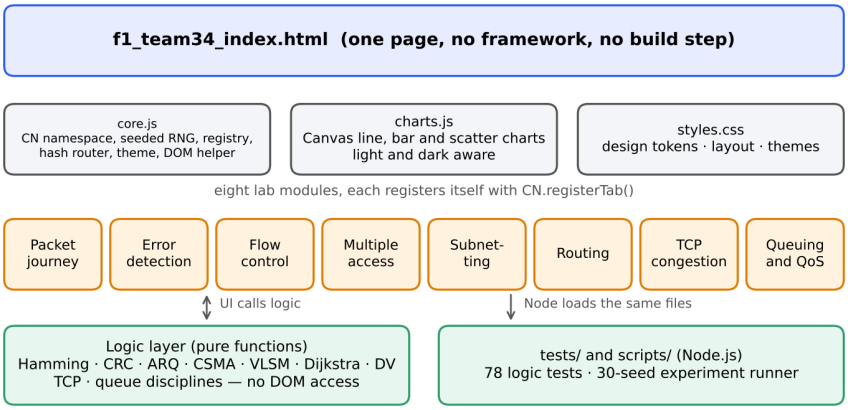


Fig 1: Architecture of Protocol Lab

4. Project Overview & Execution

A browser simulator with eight interactive labs that follow the Computer Networks syllabus: packet journey, error detection, flow control, multiple access, subnetting, routing, TCP congestion and queuing with QoS. Each lab runs a baseline and an improved mechanism on the same input (for example Go-Back-N against Selective Repeat) and the results are checked by 78 automated tests.

Run Instructions:

```
git clone https://github.com/savir18/CN.git
cd CN
open f1_team34_index.html        # double-click the file; no install, no server
node tests/f1_team34_tests.js    # optional: 78 passed, 0 failed
node scripts/f1_team34_experiments.js   # optional: 30-seed result tables
```

Demo Script / Verification Steps:

1. Open f1_team34_index.html. The landing page lists the eight labs. Click Follow a packet and step through the Packet journey: headers are added layer by layer and the TTL falls by one at each router.
2. Open Flow control and keep the default (10 frames, window 4, frame 2 lost). Go-Back-N resends 4 frames; switch to Selective Repeat and it resends 1. Both finish at time 20.1; Stop-and-Wait finishes at 55.1.
3. Open Routing in Link state mode on the six-router network and press Next step. Dijkstra from A finalises B (7), C (9), F (11), then D and E (20).
4. In Routing choose Distance vector, the network Three in a line, run until stable, select link B to C, press Break link and run until stable again. Plain distance vector needs 31 rounds to reach infinity. Change the fix to Split horizon and repeat: it settles in 2 rounds.
5. Open TCP congestion, Window lab, default scenario. A triple duplicate ACK at round 9 drops Tahoe to 1 and Reno to half. Readout: Tahoe 245 segments, Reno 304 (24% ahead).
6. Open Queuing and QoS. Under FIFO the voice call has a mean delay of 160.8 ms and 6.60% loss. Click Priority queuing (5.64 ms, 0%) and Weighted fair queuing (7.18 ms, 0%).
7. In a terminal run node tests/f1_team34_tests.js (78 passed, 0 failed) and node scripts/f1_team34_experiments.js to reproduce the mean ± SD tables in the report.